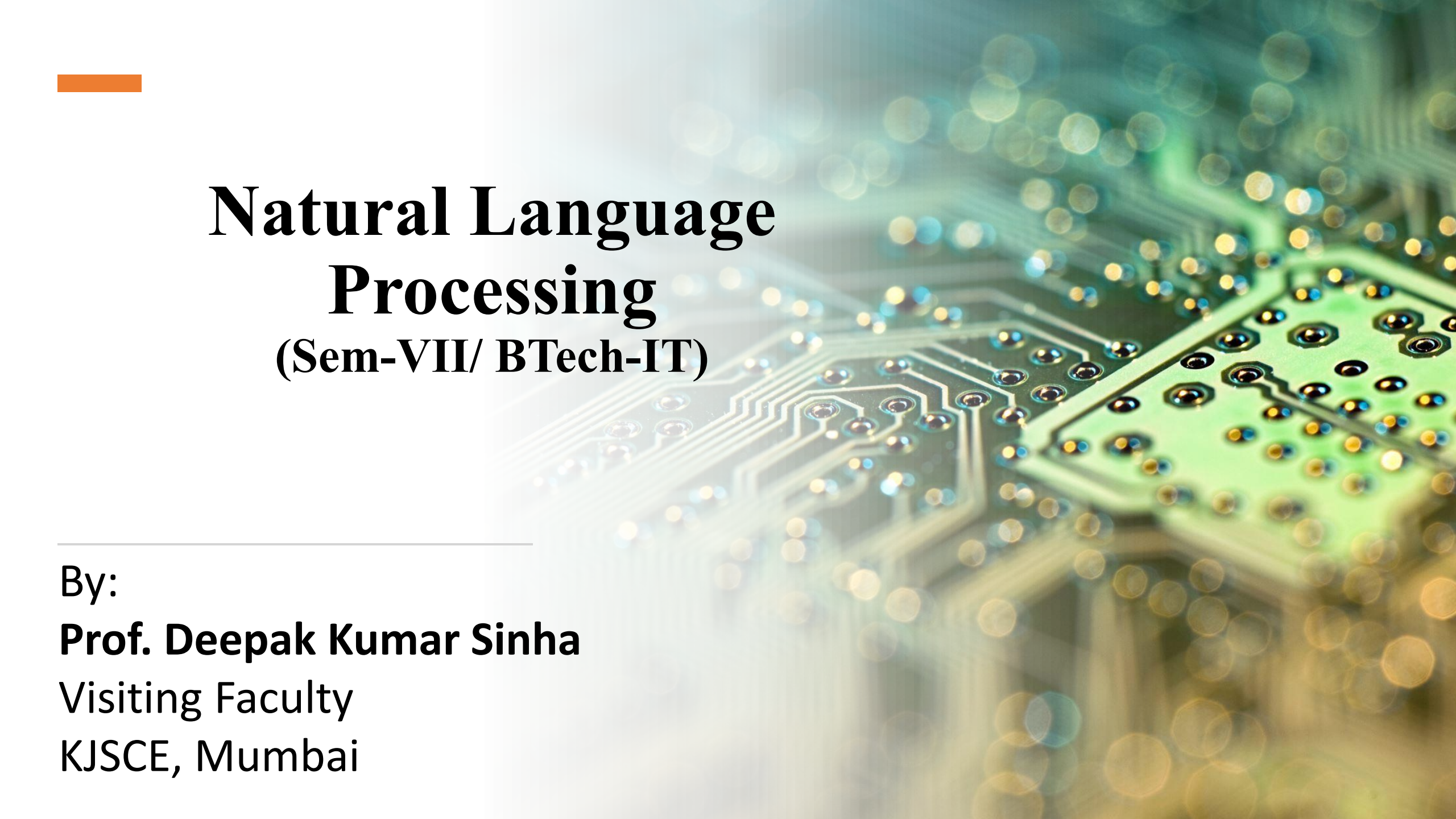
Natural Language Processing

(Sem-VII/ BTech-IT)



By:

Prof. Deepak Kumar Sinha

Visiting Faculty

KJSCE, Mumbai

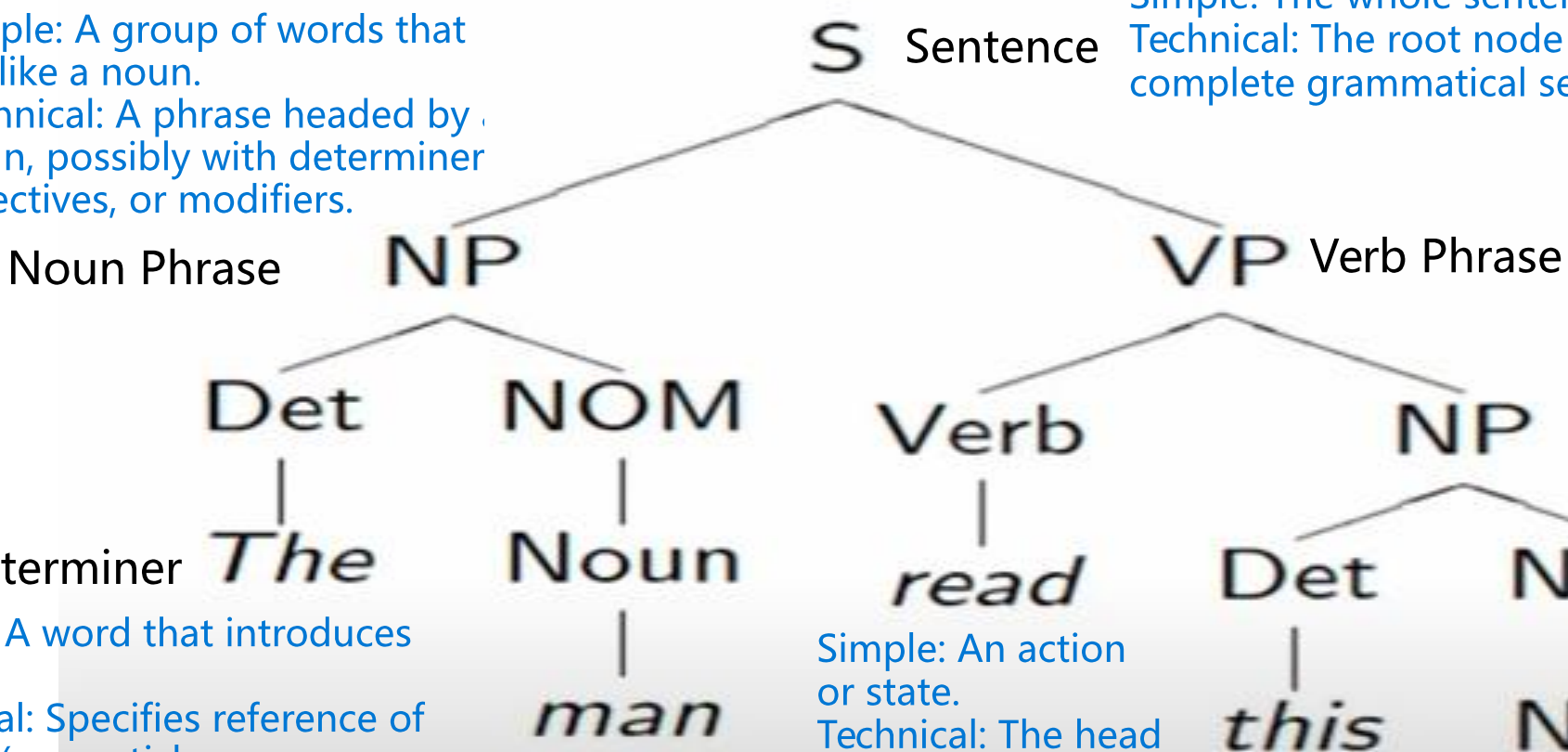
Unit-3-Structures

3.	**Structures**		**10**	**CO3**
	3.1	Theories of Parsing, Parsing Algorithms		
	3.2	Robust and Scalable Parsing on Noisy Text as in Web documents		
	3.3	Hybrid of Rule Based and Probabilistic Parsing, Scope Ambiguity and Attachment Ambiguity resolution		

What is Syntax?

- Refers to the way words are arranged together, and the relationship between then.
- **Language Models:** Importance of modeling word order
- **POS categories:** An equivalence class for words
- More complex notions: constituency, grammatical relations, subcategorization etc.

Syntax Tree: Example



Simple: A group of words that act like a noun.
Technical: A phrase headed by a noun, possibly with determiners, adjectives, or modifiers.

Simple: The whole sentence.
Technical: The root node representing a complete grammatical sentence.

Simple: The part of the sentence that tells what the subject does.
Technical: A phrase headed by a verb, possibly followed by objects or complements.

Determiner

Simple: A word that introduces a noun.
Technical: Specifies reference of a noun (e.g., articles, demonstratives, possessives).

Simple: A person, place, thing, or idea.
Technical: The head of a nominal phrase.

Simple: An action or state.
Technical: The head of a verb phrase.

Nominal

Simple: The core noun part of a noun phrase.
Technical: A sub-part of NP that contains the noun and its modifiers.

Defining the notions: Constituency

Constituent

A group of words acts as a single unit - phrases, clauses etc.

Part of Speech - "Substitution Test"

The {sad, intelligent, green, fat, ...} one is in the corner.

Constituency: Noun Phrase

- *Kermit the frog*
- *they*
- *December twenty-sixth*
- *the reason he is running for president*

Constituent Phrases

Usually named based on the word that heads the constituent:

the man from Amherst	is a Noun Phrase (NP) because the head man is a noun
extremely clever	is an Adjective Phrase (AP) because the head clever is an adjective
down the river	is a Prepositional Phrase (PP) because the head down is a preposition
killed the rabbit	is a Verb Phrase (VP) because the head killed is a verb

Words can also act as phrases

Joe grew potatoes

Joe and *potatoes* are both nouns and noun phrases

Compare with: *The man from Amherst grew beautiful russet potatoes.*

Joe appears in a place that a larger noun phrase could have been.

Evidence that constituency exists

They appear in similar environments

Kermit the frog comes on stage
They come to Massachusetts every summer
December twenty-sixth comes after Christmas
The reason he is running for president comes out only now.

But not each individual word in the consituent

**The comes our...* **is comes out...* **for comes out...*

Can be placed in a number of different locations

Consituent = Prepositional phrase: *On December twenty-sixth*

On December twenty-sixth I'd like to fly to Florida.
I'd like to fly on December twenty-sixth to Florida.
I'd like to fly to Florida on December twenty-sixth.

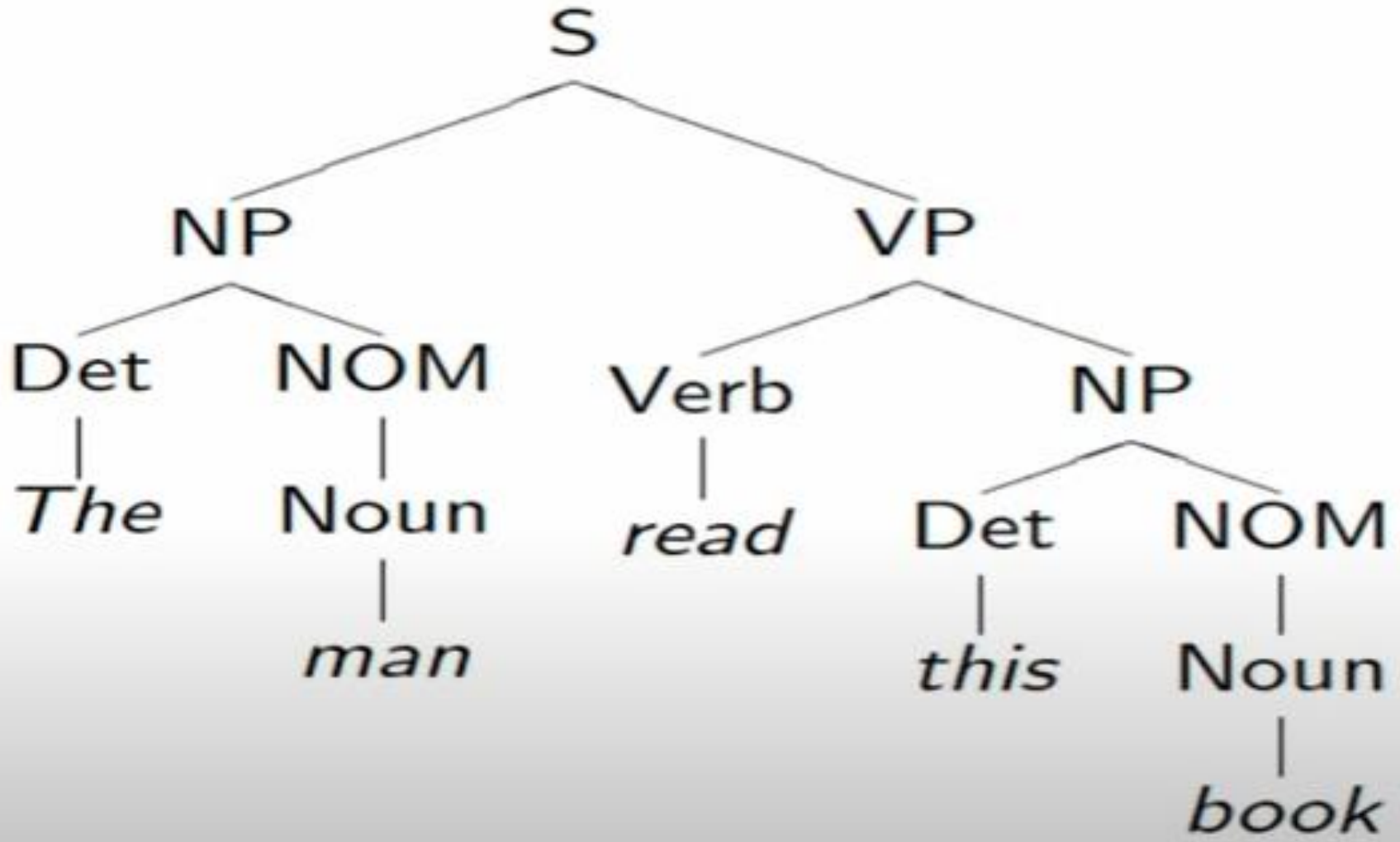
But not split apart

**On December I'd like to fly twenty-sixth to Florida.*
**On I'd like to fly December twenty-sixth to Florida.*

All have a group of words before "comes" that act as a unit (a constituent).
But if you try to move just one word from that group, like: "The comes out..." or "is comes out..."
It sounds wrong! That shows individual word don't work alone—they need their team.

A constituent is like a puzzle piece that fits into a sentence. If you try to break it or move just part of it, the sentence stops making sense.

Modeling Constituency: what tool do we need?



Modeling Constituency

Context-free grammar

The most common way of modeling constituency

Consists of production Rules

non-terminals and terminals

These rules express the ways in which the symbols of the language can be grouped and ordered together

Example

Noun phrase can be composed of either a ProperNoun or a determiner (Det) followed by a Nominal; a Nominal can be more than one nouns

$NP \rightarrow$ Det Nominal

$NP \rightarrow$ ProperNoun

$Nominal \rightarrow$ Noun | Noun Nominal

NP → Det Nominal: A noun phrase can be a determiner followed by a nominal (e.g., “the cat”).
NP → ProperNoun: A noun phrase can be a single proper noun (e.g., “Mumbai”).
Nominal → Noun | Noun Nominal: A nominal can be one noun or a chain of nouns, enabling multi-noun compounds (e.g city road construction”).

CFG for Languages

CFG: $G = (T, N, S, R)$

- T: set of terminals
  These are the actual words or symbols in the language. In NLP, terminals are typically words like "dog "runs", "Mumbai".
- N: set of non-terminals
  These are syntactic categories like Sentence (S), Noun Phrase (NP), Verb Phrase (VP), etc. They help define the structure of the language.
  - For NLP, we distinguish out a set $P \subset N$ of pre-terminals, which always rewrite as terminals
    Special subset of non-terminals that directly rewrite to terminals. They usually correspond to Part-of-Speech (POS) tags like: Noun, Verb, Determiner
- S : start symbol
  The starting point of parsing or generation. Usually S for Sentence.
- R: Rules/productions of the form $X \rightarrow \gamma$, $X \in N$ and $\gamma \in (T \cup N)*$
  rewrite rules

Terminals and pre-terminals

Terminals mainly correspond to words in the language while pre-terminals mainly correspond to POS categories

A context-free grammar consists of a set of rules or productions, each of which expresses the ways that symbols of the language can be grouped and ordered together, Lexicon and a lexicon of words and symbols.

CFG as a generator

$NP \rightarrow$ Det Nominal
$NP \rightarrow$ ProperNoun
$Nominal \rightarrow$ Noun | Noun Nominal
$Det \rightarrow$ a
$Det \rightarrow$ the
$Noun \rightarrow$ flight
Generating 'a flight':
$NP \rightarrow$ Det Nominal
$\rightarrow$ Det Noun $\rightarrow$ a Noun $\rightarrow$ a flight

- Thus a CFG can be used to randomly generate a series of strings
- This sequence of rule expansions is called a derivation of the string of words, usually represented as a tree

The emphatic form is created by using the auxiliary verb "do" (in its present form "do" or "does" and past form "did") combined with the base form of the main verb. This structure is often used to add emphasis to a statement, particularly in response to a negative assertion or to clarify a point.

Grammar Rewrite Rules

Auxiliary-led sentences (questions or emphatic forms).

S → NP VP
S → Aux NP VP
S → VP
NP → Det NOM
NOM → Noun
NOM → Noun NOM
VP → Verb
VP → Verb NP

Det → *that* | *this* | *a* | *the*
Noun → *book* | *flight* | *meal* | *man*
Verb → *book* | *include* | *read*
Aux → *does*

Note on homonymy: "book" appears both as a Verb ("to book a flight") and a Noun ("a book"), showing how the same word can belong to different POS categories depending on context.

S → NP VP
→ Det NOM VP
→ *The* NOM VP
→ *The* Noun VP
→ *The man* VP
→ *The man* Verb NP
→ *The man read* NP
→ *The man read* Det NOM
→ *The man read this* NOM
→ *The man read this* Noun
→ *The man read this book*

What is Parsing?

- The process of taking a string and a grammar and returning all possible parse trees for that string
- That is, find all trees, whose root is the start symbol S, which cover exactly the words in the input

What are the constraints? "book that flight"

- There must be three leaves, *book*, *that* and *flight*
- The tree must have one root, the start symbol S
- Give rise to two search strategies: *top-down* (goal-oriented) and *bottom-up* (data-directed)

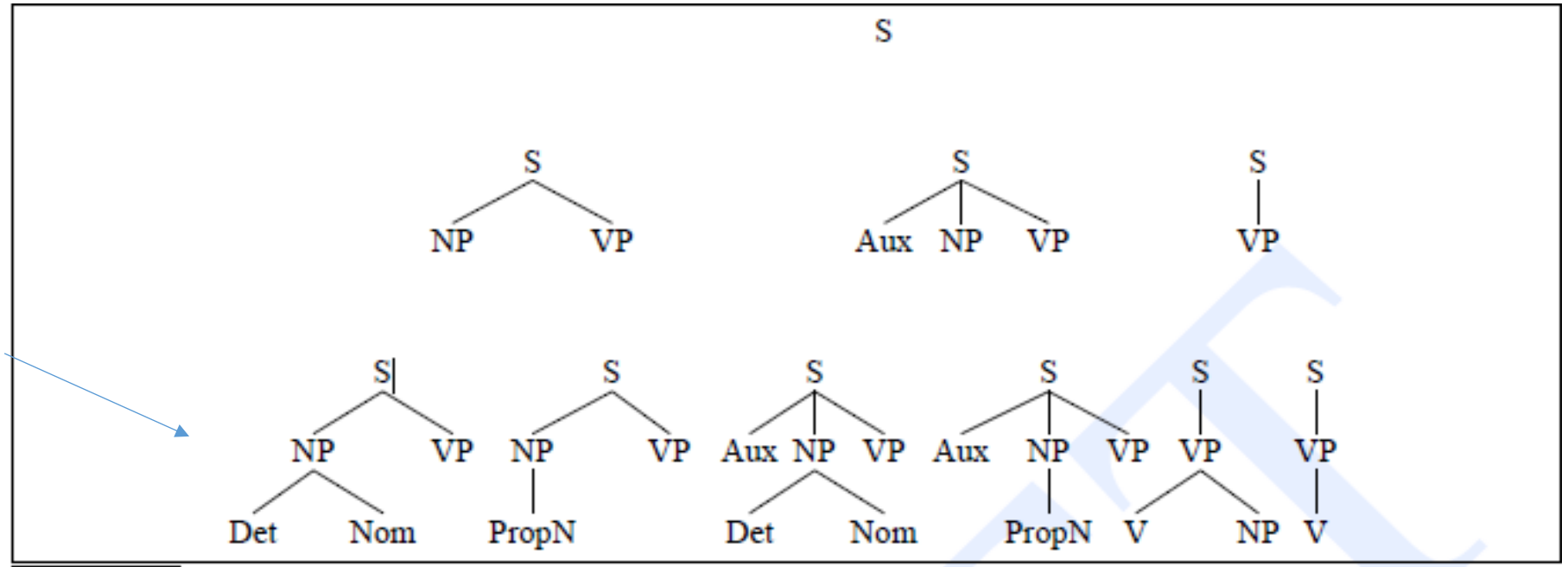
Top-Down Parsing

- Searches for a parse tree by trying to build upon the root node S down to the leaves
- Start by assuming that the input can be derived by the designated start symbol S
- Find all trees that can start with S, by looking at the grammar rules with S on the left-hand side
- Trees are grown downward until they eventually reach the POS categories at the bottom

Grammar Rewrite Rules

S → NP VP
S → Aux NP VP
S → VP
NP → Det NOM
NOM → Noun
NOM → Noun NOM
VP → Verb
VP → Verb NP

Det → *that* | *this* | *a* | *the*
Noun → *book* | *flight* | *meal* | *man*
Verb → *book* | *include* | *read*
Aux → *does*

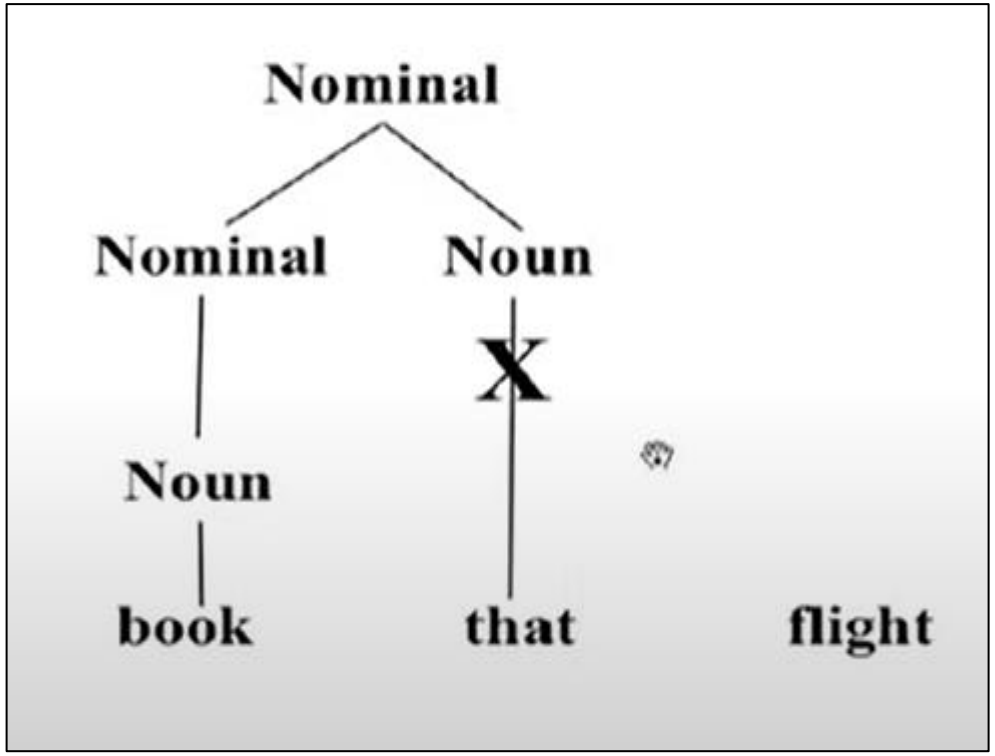


Bottom-Up Parsing

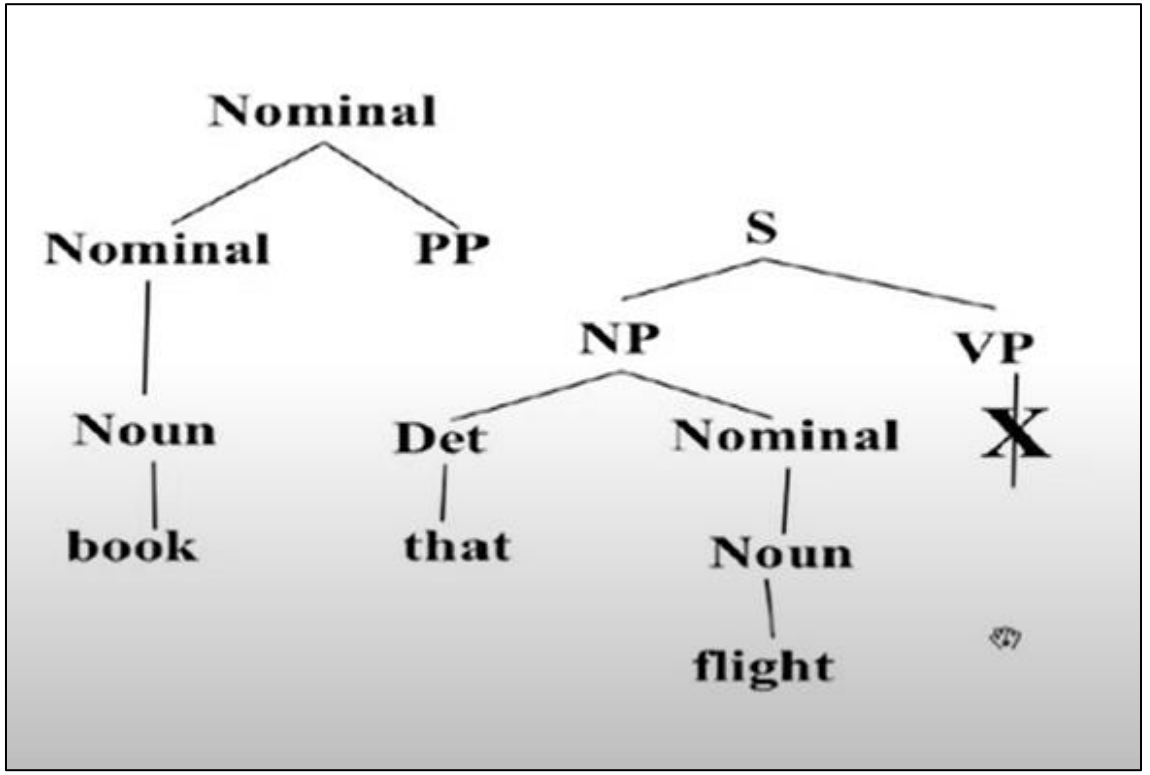
- The parser starts with the words of the input, and tries to build trees from the words up, by applying rules from the grammar one at a time
- Parser looks for the places in the parse-in-progress where the right-hand-side of some rule might fit.

Bottom-Up Parsing

Raw Lexical Categories

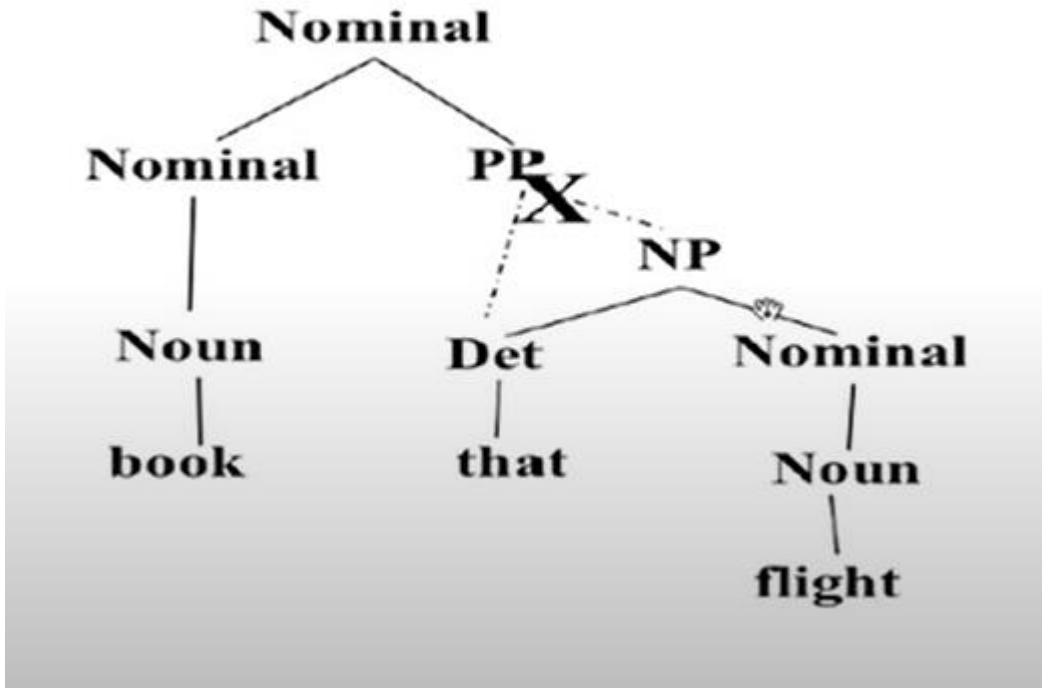


Early Phrase Structure

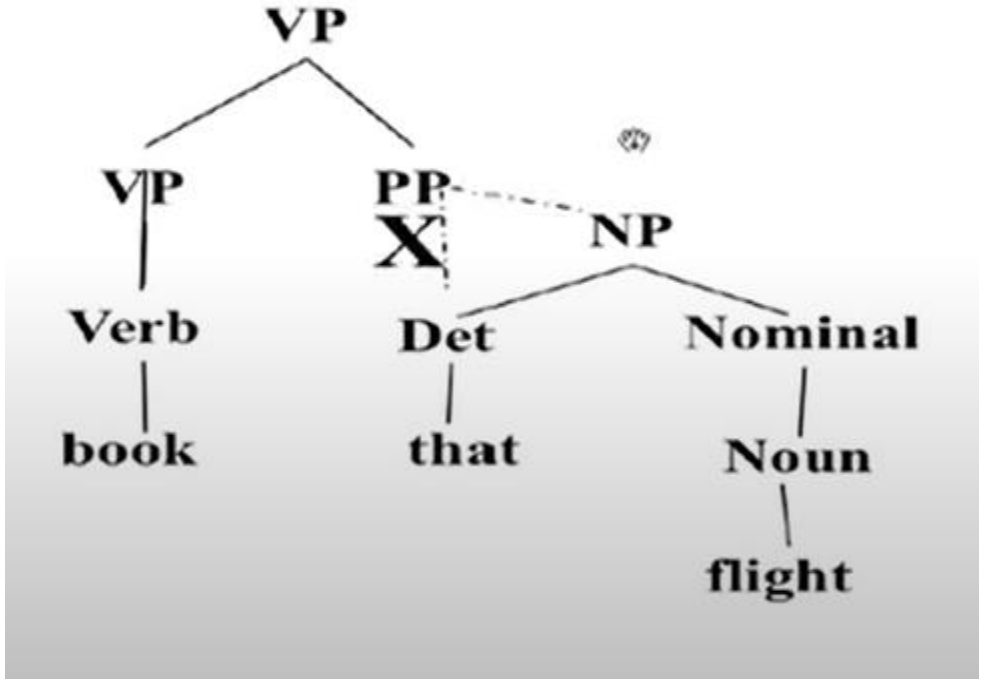


Bottom-Up Parsing

Reinterpreting "that"

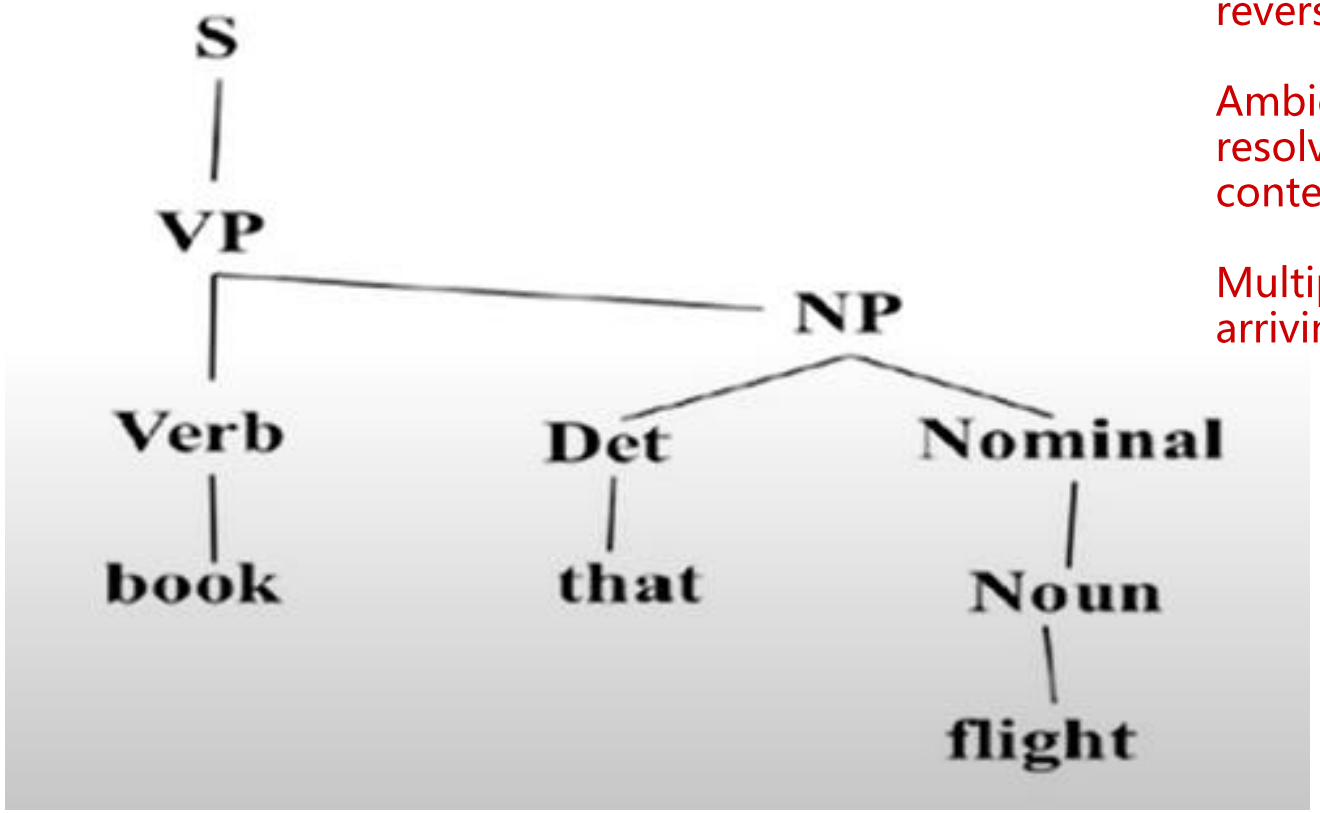


Correct Verb Interpretation



Bottom-Up Parsing

Final Structure

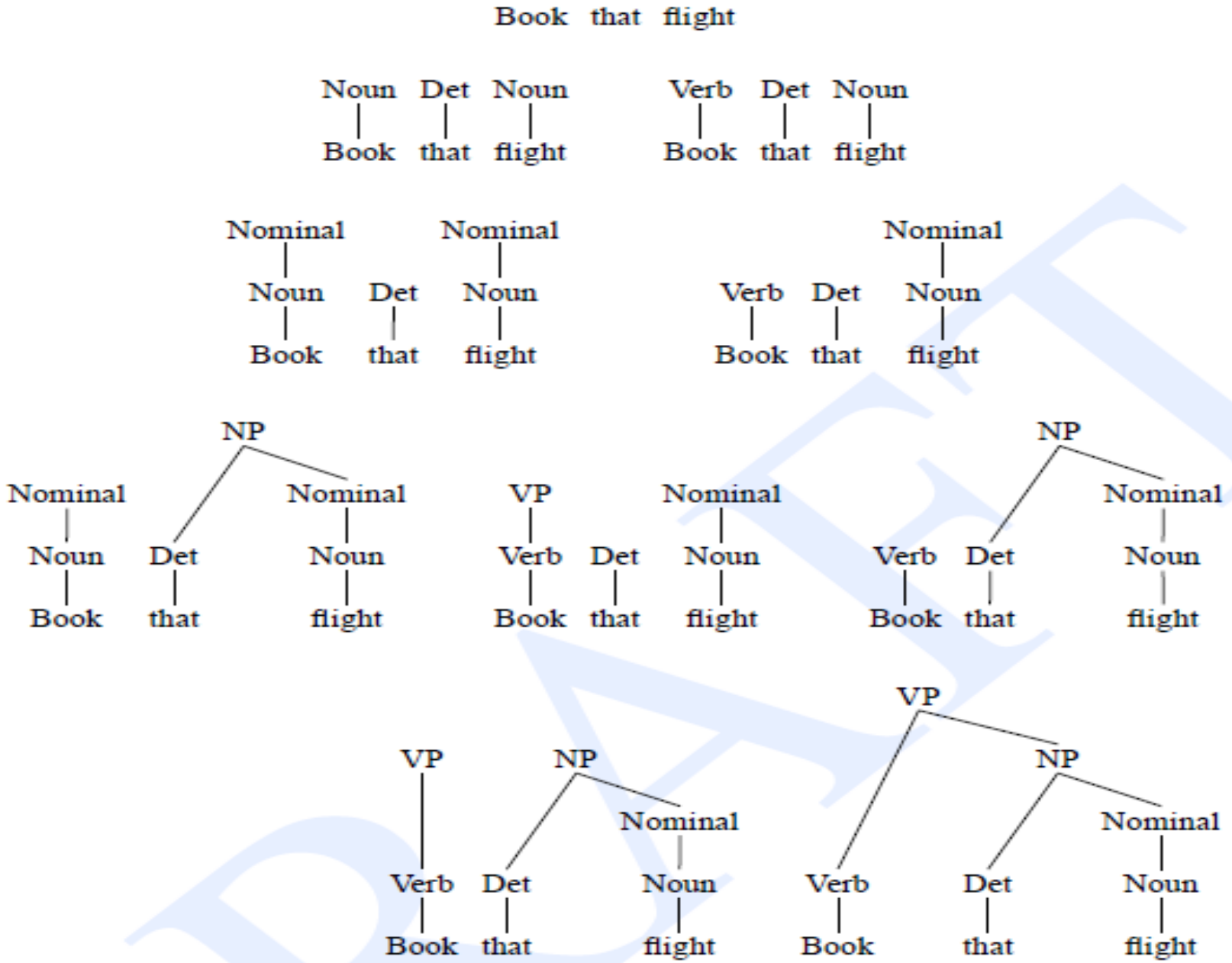


Bottom-Up Parsing builds from words to sentence by applying grammar rules in reverse.

Ambiguity (e.g., "book" as noun vs. verb) resolved through rule application and context.

Multiple parse paths are explored before arriving at the correct one.

Bottom up


Book that flight
Noun Det Noun
Book that flight
Verb Det Noun
Book that flight
Nominal
Noun
Book
Det
that
Nominal
Noun
flight
Nominal
Verb
Book
Det
that
Nominal
Noun
flight
NP
Nominal
Noun
Book
Det
that
Nominal
Noun
flight
VP
Verb
Book
Det
that
Nominal
Noun
flight
NP
Verb
Book
Det
that
Nominal
Noun
flight
VP
Verb
Book
NP
Det
that
Nominal
Noun
flight
VP
Verb
Book
NP
Det
that
Nominal
Noun
flight

Top-Down vs. Bottom-Up

☑ Pros:
Efficient in avoiding dead ends that can't form a complete sentence.
Good for goal-directed parsing.

✗ Cons:
May waste time exploring grammar paths that don't match t actual input.
Example: It might try to parse "book that flight" as a noun phrase when it's actually a verb phrase.

☑ Pros:
Always grounded in the actual input — builds only from what's present.
Avoids speculative expansions.

✗ Cons:
Might build partial structures that can't be completed into a full sentence.
Example: It might treat "book" as a noun and build a noun phrase which doesn't lead to a valid sentence.

- Top down never explores options that will not lead to a full parse, but can explore many options that never connect to the actual sentence.
- Bottom up never explores options that do not connect to the actual sentence but can explore options that can never lead to a full parse.
- Relative amounts of wasted search depend on how much the grammar branches in each direction.

If your grammar has lots of top-down rules, top-down parsing might waste more effort.
If your grammar has many bottom-up combinations, bottom-up parsing might build many dead-end structures.

Dynamic programming parsing is a technique used to efficiently parse sentences by avoiding redundant work. It's especially useful when the grammar is complex and recursive.

Dynamic Programming Parsing

- To avoid extensive repeated work, must cache intermediate results, i.e. completed phrases.
- Caching (memoizing) critical to obtaining a polynomial time parsing (recognition) algorithm for CFGs.
- Dynamic programming algorithms based on both top-down and bottom-up search can achieve $O(n^3)$ recognition time where n is the length of the input string.

Memoization = storing results of expensive function calls and returning the cached result when the same inputs occur again.

Without caching, parsing can take exponential time in worst cases.

Recognition = deciding whether a string belongs to the language defined by the grammar.

Dynamic Programming Parsing Methods

- CKY (Cocke-Kasami-Younger) algorithm: bottom-up, requires normalizing the grammar

  Requires the grammar to be in Chomsky Normal Form (CNF) — meaning every rule is either:
  A → B C (two non-terminals)
  A → a (a single terminal)

- Earley Parser - top-down, does not require normalizing grammar, more complex

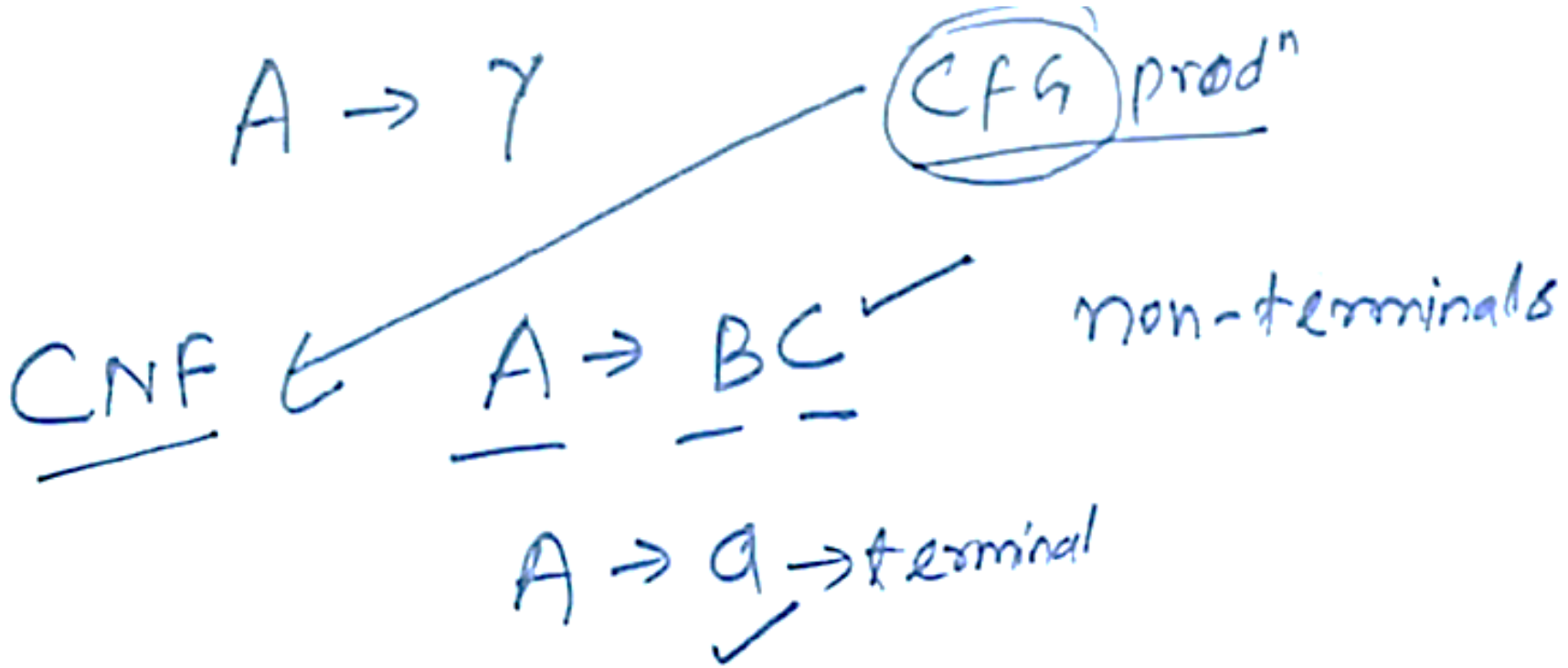
  Does NOT require grammar normalization — works with any CFG.

- More generally, *chart parsers* retain completed phrases in a chart and can combine top-down and bottom-up searches.

CKY Algorithm

- Grammar must be converted to Chomsky normal form (CNF) in which all productions must have
  - Either, exactly two non-terminals on the RHS
  - Or, 1 terminal symbol on the RHS
- Parse bottom-up storing phrases formed from all substrings in a triangular table (chart)

CKY Algorithm



Here Y can be a sequence of terminals and non-terminals
B and C are non-terminals
a is a non-terminal.

Converting to CNF

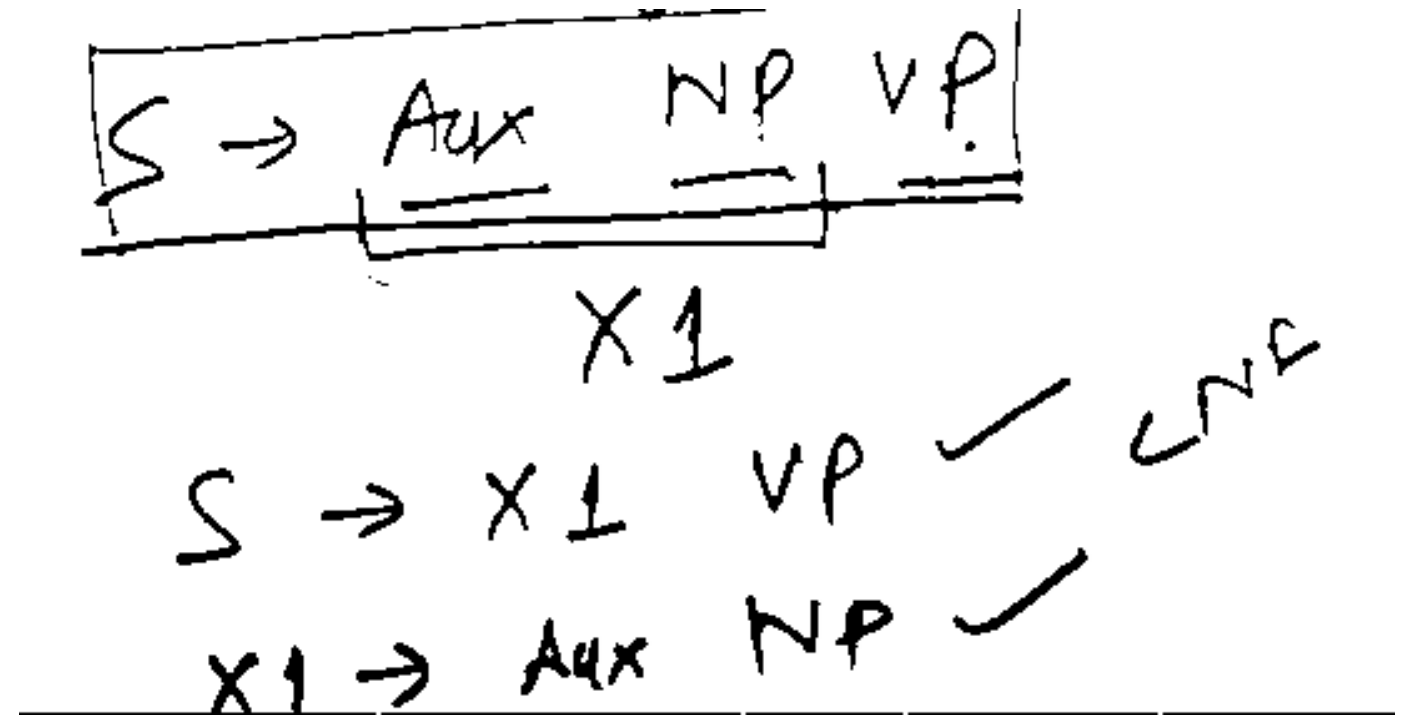
Original Grammar

S → NP VP
S → Aux NP VP
S → VP
NP → Pronoun
NP → Proper-Noun
NP → Det Nominal
Nominal → Noun
Nominal → Nominal Noun
Nominal → Nominal PP
VP → Verb
VP → Verb NP
VP → VP PP
PP → Prep NP
Pronoun → I | he | she | me
Noun → book | flight | meal | money
Verb → book | include | prefer
Proper-Noun → Houston | NWA

Chomsky Normal Form

S → NP VP
S → X1 VP
X1 → Aux NP
S → book | include | prefer
S → Verb NP
S → VP PP
NP → I | he | she | me
NP → Houston | NWA
NP → Det Nominal
Nominal → book | flight | meal | money
Nominal → Nominal Noun
Nominal → Nominal PP
VP → book | include | prefer
VP → Verb NP
VP → VP PP
PP → Prep NP
Pronoun → I | he | she | me
Noun → book | flight | meal | money
Verb → book | include | prefer
Proper-Noun → Houston | NWA

Converting to CNF



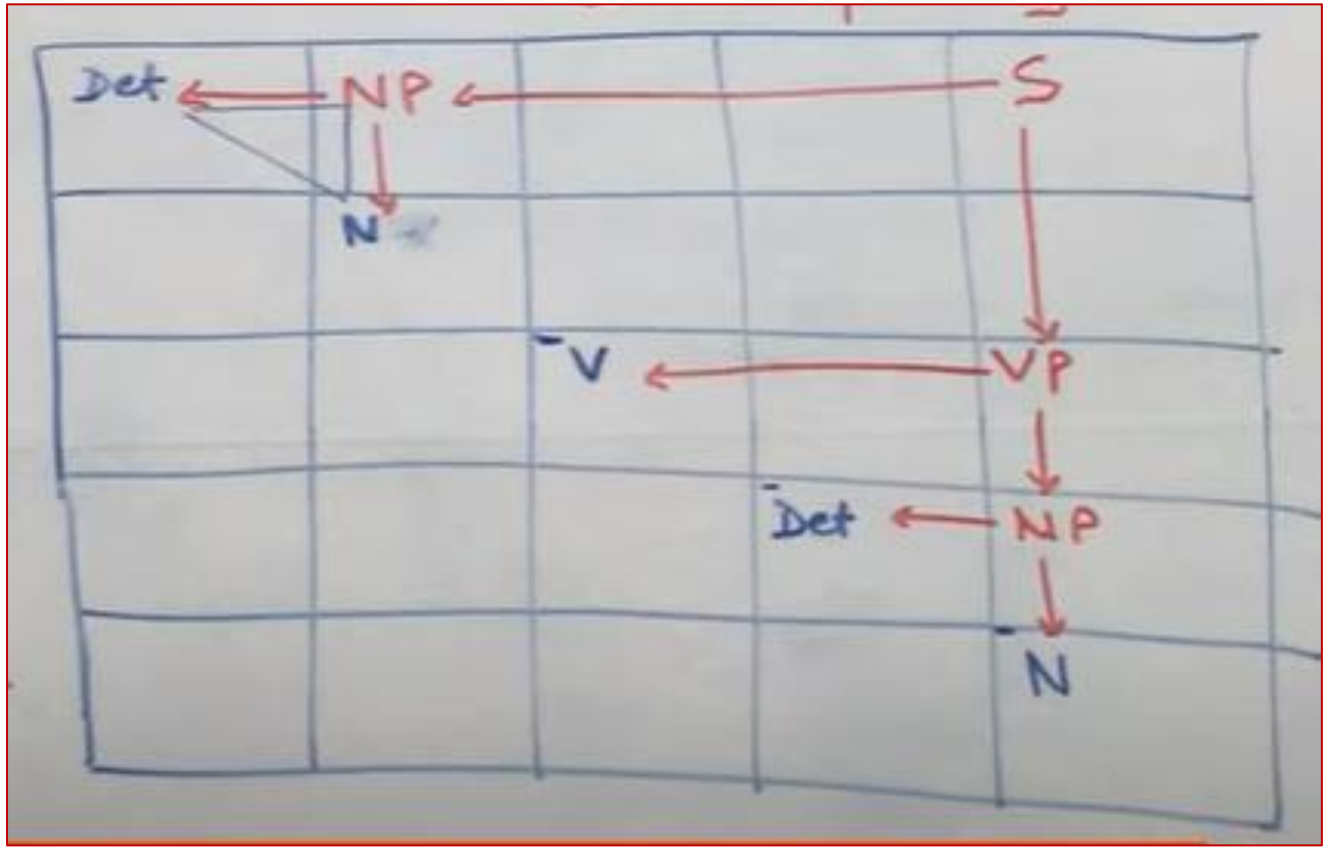
CKY Algorithm

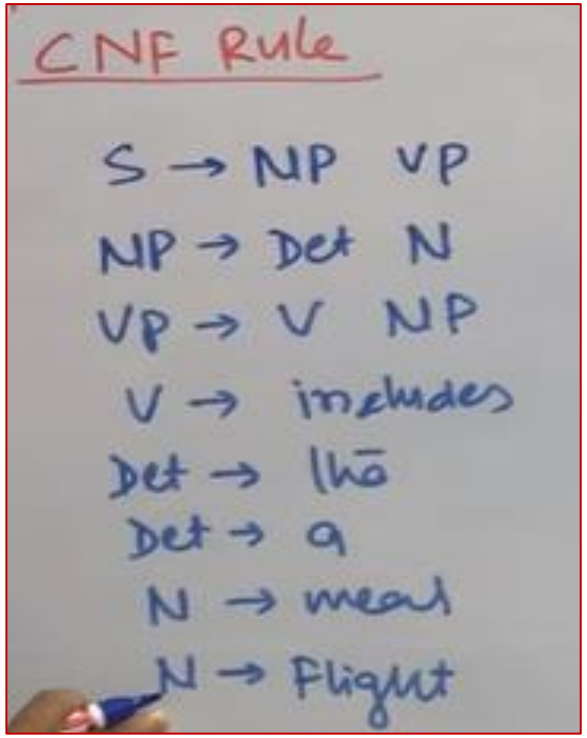
- Let n be the number of words in the input. Think about $n + 1$ lines separating them, numbered 0 to n.
- x_{ij} will denote the words between line i and j
- We build a table so that x_{ij} contains all the possible non-terminal spanning for words between line i and j.
- We build the Table bottom-up.

a 1	pilot 2	likes 3	flying 4	planes 5
DT	NP	-	-	S S
	NN	-	-	-
		VBZ	-	VP VP
			JJ VBG	NP VP
				NNS

$S \rightarrow NP\ VP$
$VP \rightarrow VBG\ NNS$
$VP \rightarrow VBZ\ VP$
$VP \rightarrow VBZ\ NP$
$NP \rightarrow DT\ NN$
$NP \rightarrow JJ\ NNS$
$DT \rightarrow a$
$NN \rightarrow pilot$
$VBZ \rightarrow likes$
$VBG \rightarrow flying$
$JJ \rightarrow flying$
$NNS \rightarrow planes$

The flight includes a meal





Few Example for practice

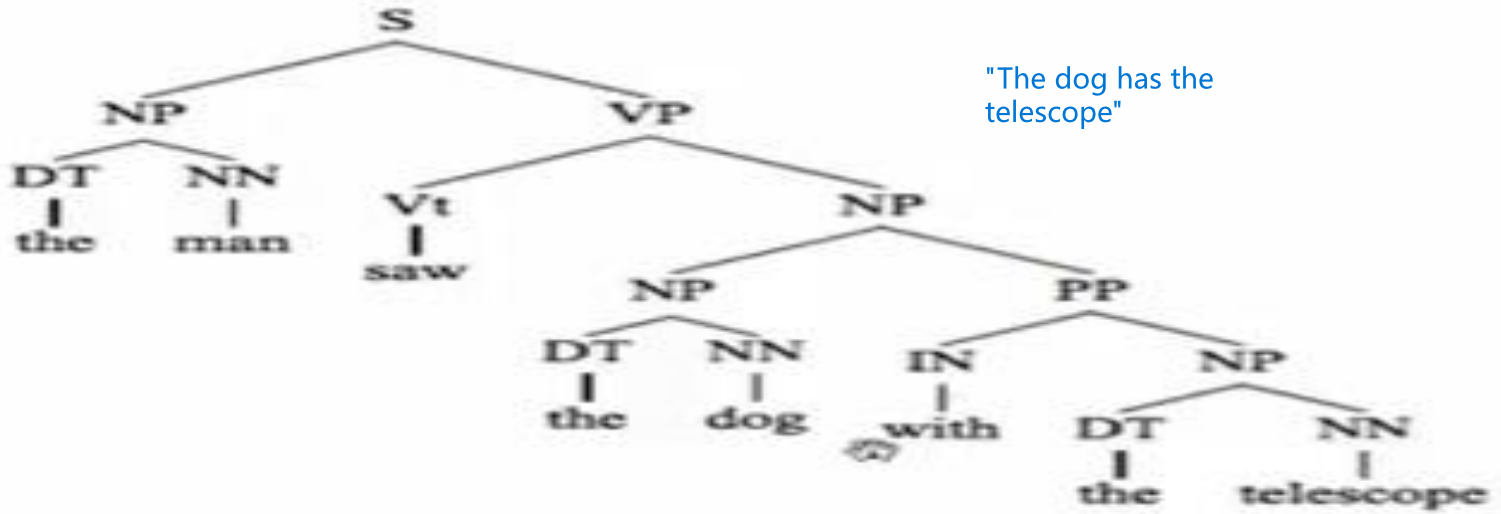
1. Solve the problem given in book
2. For the given grammar, check the acceptance of string w = baaba using CYK Algorithm-

- S → AB / BC
- A → BA / a
- B → CC / b
- C → AB / a

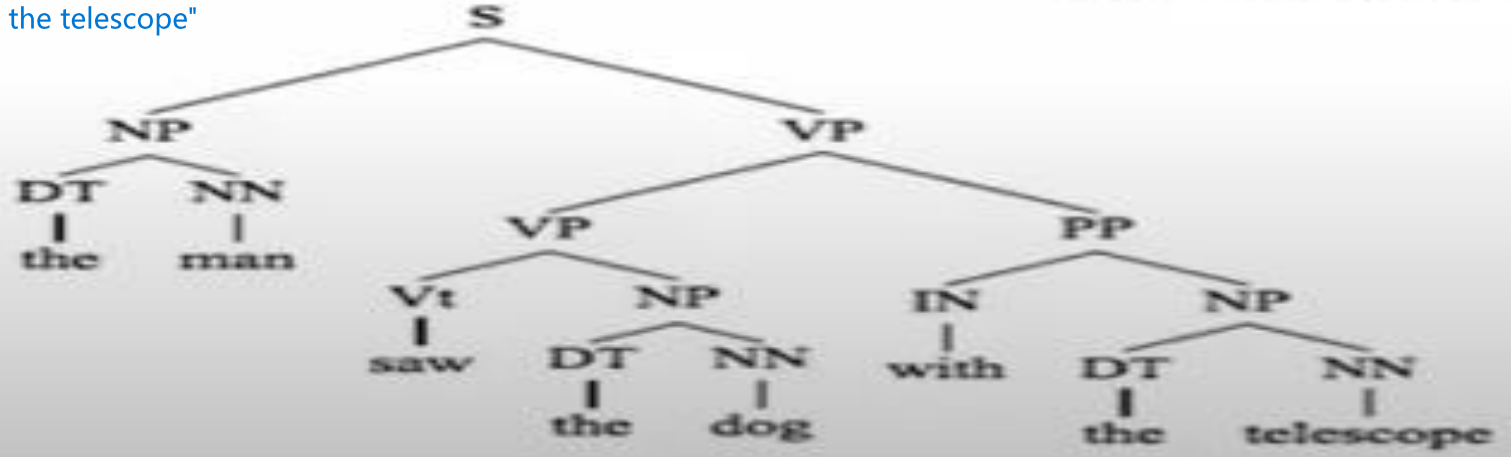
- Show the CYK Algorithm with the following example:
  - CNF grammar **G**
    - S → AB | BC
    - A → BA | a
    - B → CC | b
    - C → AB | a
  - **w** is ababa
  - Question Is **ababa** in L(G)?

What about Ambiguities?

"The dog has the telescope"



"The man used the telescope"



Machines must choose the correct parse based on context, semantics, or world knowledge.
Ambiguity like this affects tasks like translation, question answering, and speech recognition.

Probabilistic Context-free grammars (PCFGs)

A PCFG is a context-free grammar where each rule has a probability. These probabilities help decide which parse tree is more likely when multiple parses are possible.

PCFG: $G = (T, N, S, R, P)$

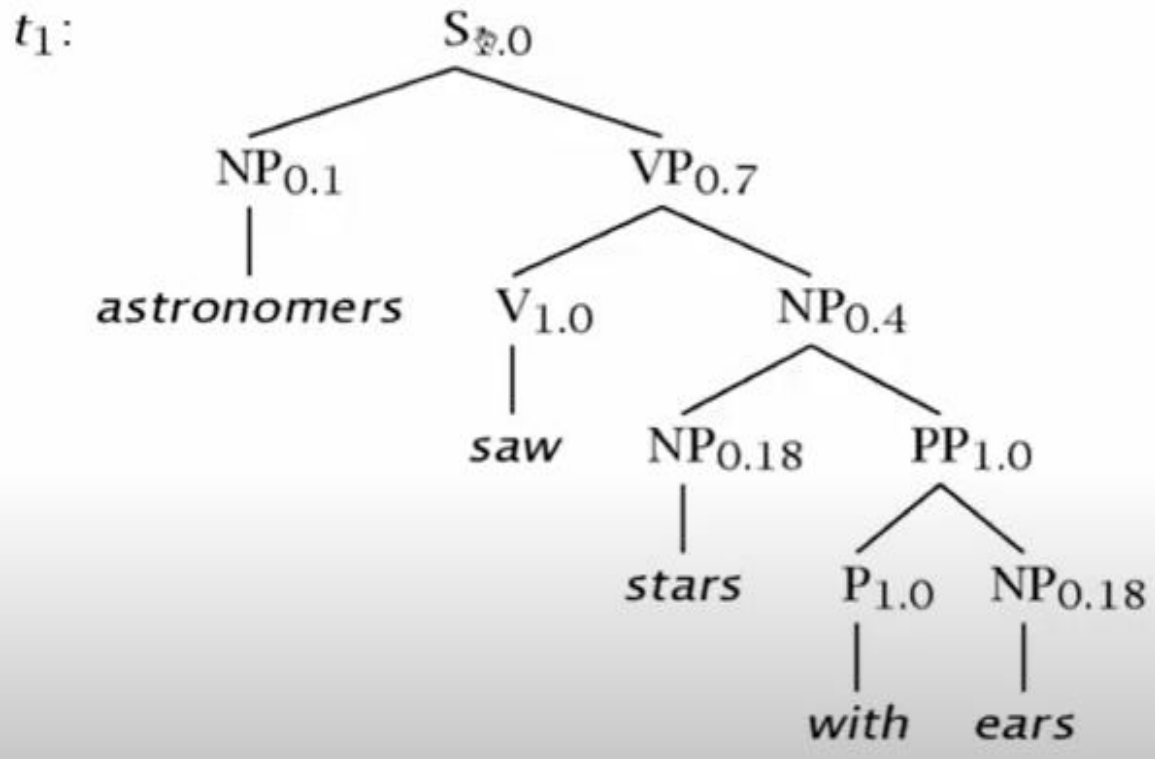
- T: set of terminals
- N: set of non-terminals
  - For NLP, we distinguish out a set $P \subset N$ of pre-terminals, which always rewrite as terminals
- S : start symbol
- R: Rules/productions of the form $X \rightarrow \gamma$, $X \in N$ and $\gamma \in (T \cup N)*$
- $P(R)$ gives the probability of each rule.

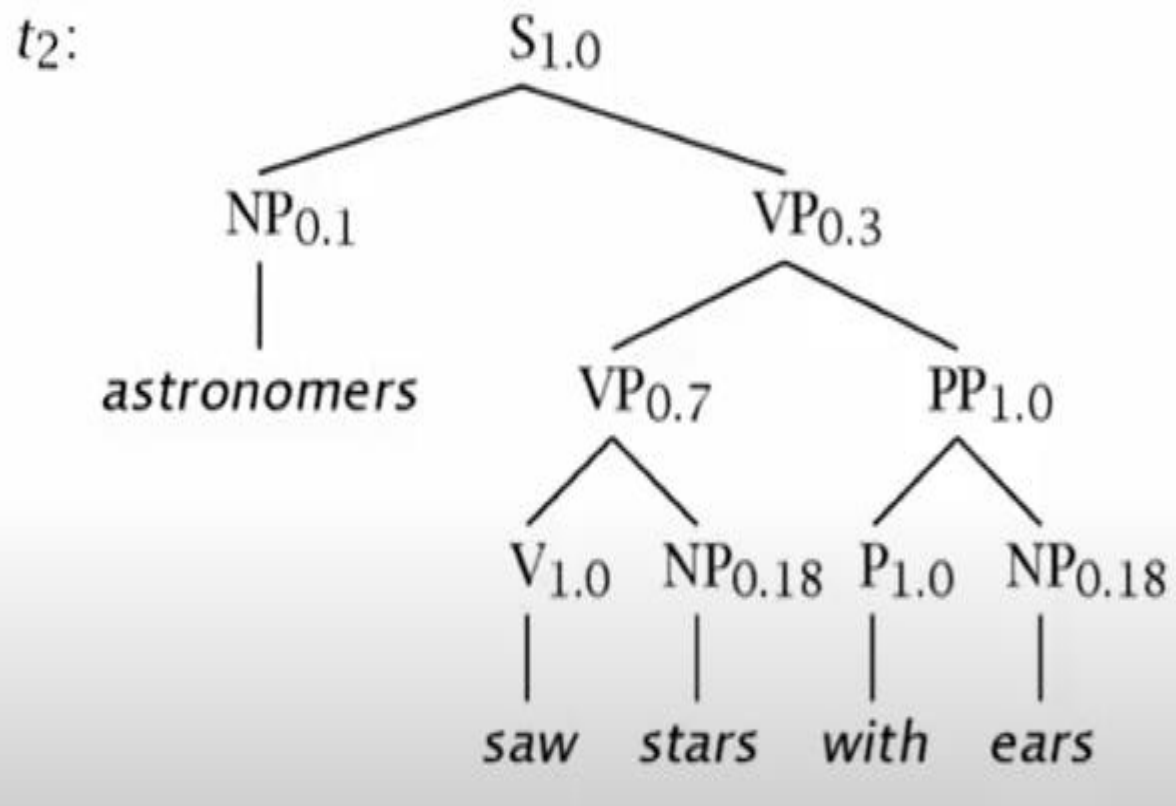
$$\forall X \in N, \sum_{X \rightarrow \gamma \in R} P(X \rightarrow \gamma) = 1$$

A Simple PCFG (in CNF)

S	→	NP VP	1.0	NP	→	NP PP	0.4
VP	→	V NP	0.7	NP	→	*astronomers*	0.1
VP	→	VP PP	0.3	NP	→	*ears*	0.18
PP	→	P NP	1.0	NP	→	*saw*	0.04
P	→	*with*	1.0	NP	→	*stars*	0.18
V	→	*saw*	1.0	NP	→	*telescope*	0.1

Example Trees





Probability of trees and strings

- $P(t)$: The probability of tree is the product of the probabilities of the rules used to generate it
- $P(w_{1n})$: The probability of the string is the sum of the probabilities of the trees which have that string as their yield

Tree and String probabilities

w_{15} = *astronomers saw stars with ears*

$P(t_1) = 1.0 * 0.1 * 0.7 * 1.0 * 0.4 * 0.18 * 1.0 * 1.0 * 0.18$

$= 0.0009072$

$P(t_2) = 1.0 * 0.1 * 0.3 * 0.7 * 1.0 * 0.18 * 1.0 * 1.0 * 0.18$

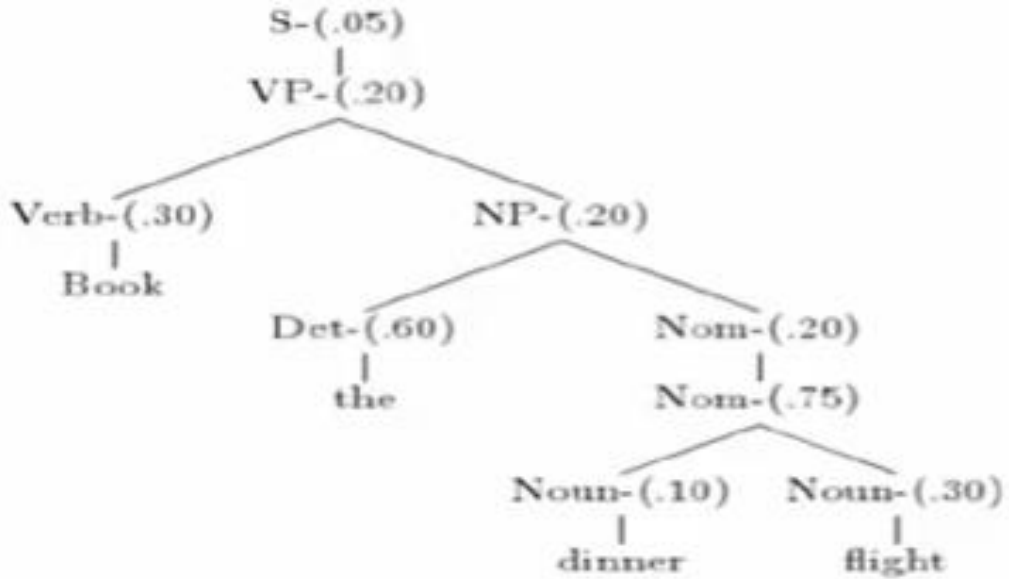
$= 0.0006804$

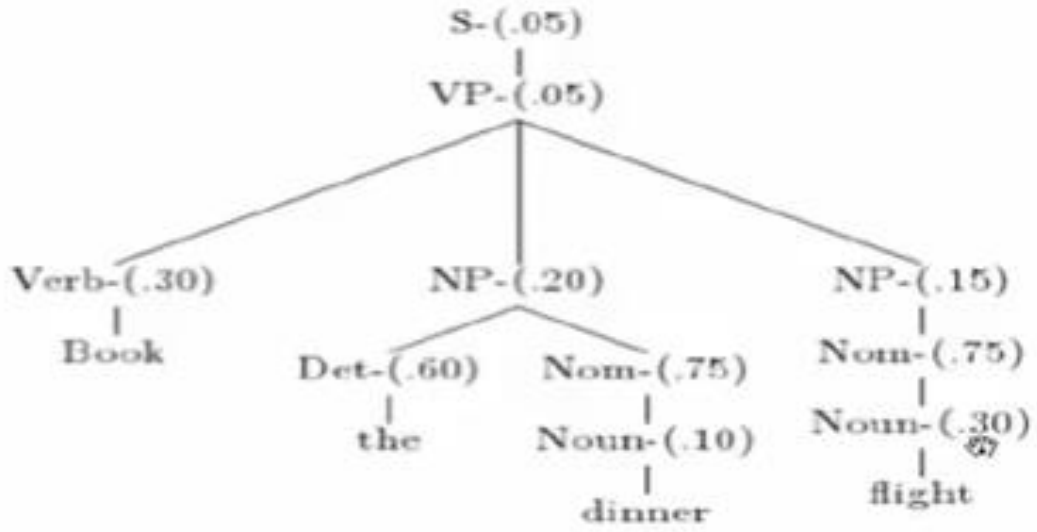
$P(w_{15}) = P(t_1) + P(t_2)$

$= 0.0009072 + 0.0006804$

$= 0.0015876$

"Book the dinner flight"





Probabilities

- Parse tree 1: $.05 \times .20 \times .30 \times .20 \times .60 \times .20 \times .75 \times .10 \times .30 = 1.62 \times 10^{-6}$
- Parse tree 2: $.05 \times .05 \times .30 \times .20 \times .60 \times .75 \times .10 \times .15 \times .75 \times .30 = 2.28 \times 10^{-7}$

Features of PCFGs

- As the number of possible trees for a given input grows, a PCFG gives some idea of the plausibility of a particular parse
- *But* the probability estimates are based purely on structural factors, and do not factor in lexical co-occurrence. Thus, PCFG does not give a very good idea of the plausibility of the sentence.
- Real text tends to have grammatical mistakes. PCFG avoids this problem by ruling out nothing, but by giving implausible sentences a low probability
- In practice, a PCFG is a worse language model for English than an n-gram model
- All else being equal, the probability of a smaller tree is greater than a larger tree

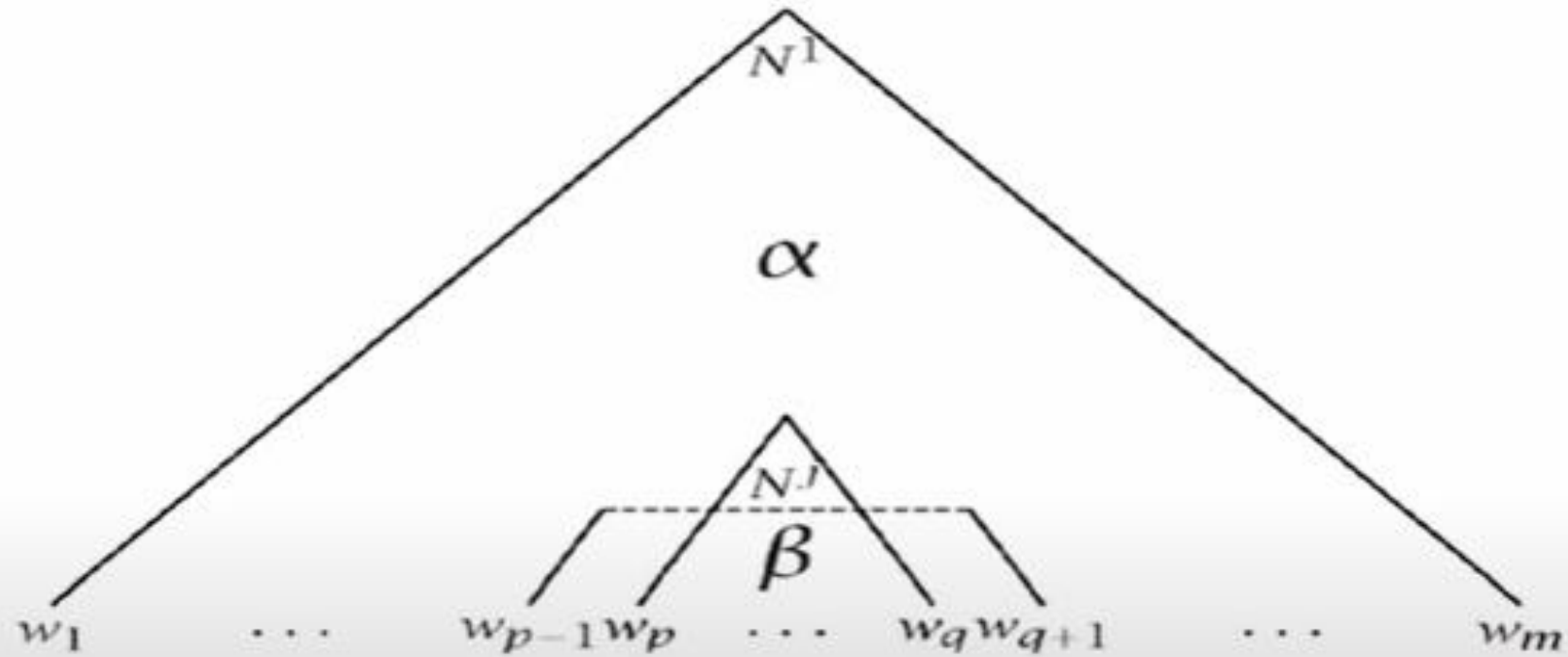
CKY for PCFG

a 1	pilot 2	likes 3	flying 4	planes 5
DT [0.3]	NP [.009]	-	-	S [1.4688×10^{-5}] S [6.12×10^{-6}]
	NN [0.1]	-	-	-
		VBZ [0.4]	-	VP [.001632] VP [.00068]
			JJ [0.1] VBG [0.5]	NP [.0136] VP [.017]
				NNS [.34]

$S \rightarrow NP\ VP$	[1.0]
$VP \rightarrow VBG\ NNS$	[0.1]
$VP \rightarrow VBZ\ VP$	[0.1]
$VP \rightarrow VBZ\ NP$	[0.3]
$NP \rightarrow DT\ NN$	[0.3]
$NP \rightarrow JJ\ NNS$	[0.4]
$DT \rightarrow a$	[0.3]
$NN \rightarrow pilot$	[0.1]
$VBZ \rightarrow likes$	[0.4]
$VBG \rightarrow flying$	[0.5]
$JJ \rightarrow flying$	[0.1]
$NNS \rightarrow planes$	[.34]

$0.009 \times 0.00068 \times 1.0 = 6.12 \times 10^{-6}$

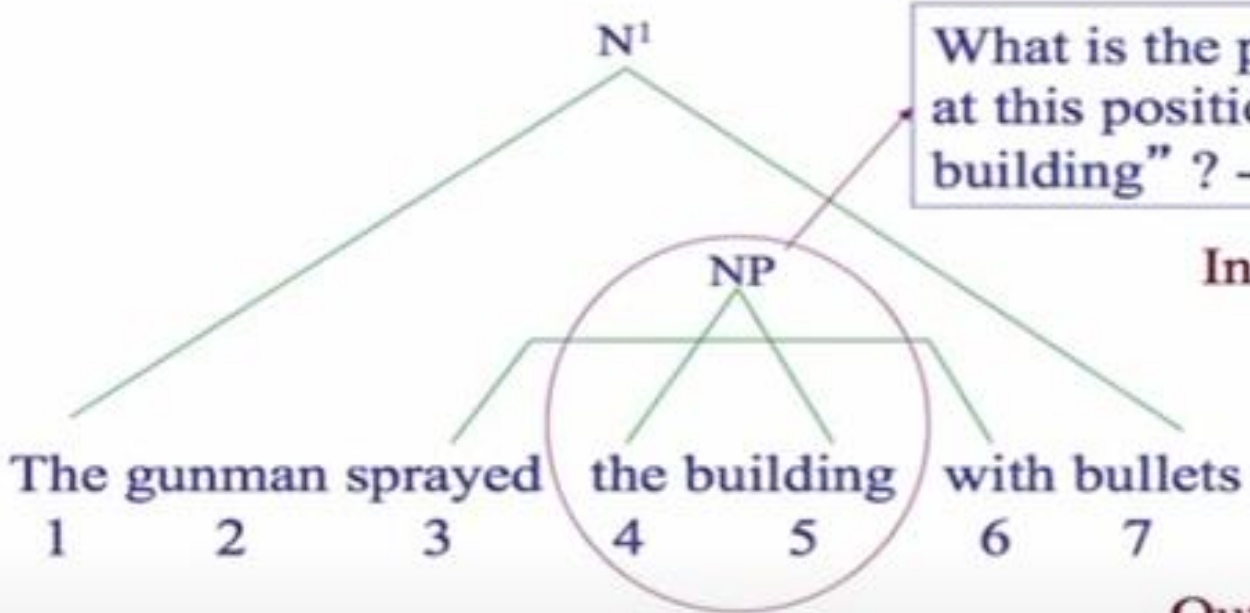
Inside and Outside Probabilities



Outside: $\alpha_j(p,q) = P(w_{1(p-1)}, N^j_{pq}, w_{(q+1)m}|G)$

Inside: $\beta_j(p,q) = P(w_{pq}|N^j_{pq}, G)$.

Inside-outside probabilities



What is the probability that an NP at this position will derive "the building" ? - $\beta_{NP}(4,5)$

Inside Probabilities

Outside Probabilities

What is the probability of starting from N^1 and deriving "The gunman sprayed", a NP and "with bullets" ? - $\alpha_{NP}(4,5)$

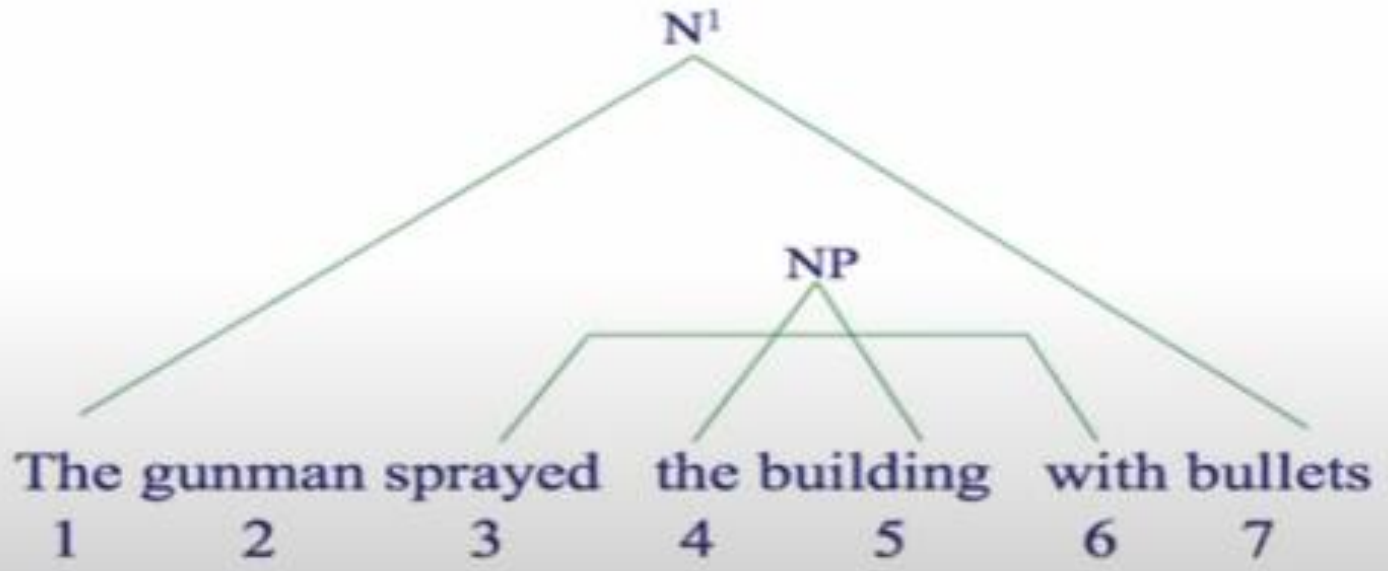
Inside-outside probabilities

Inside targets the content “the building”; outside targets the surrounding context plus a placeholder NP at (4, 5).

$$\alpha_{NP}(4,5) \text{ for "the building"}$$

$$= P(\text{The gunman sprayed}, NP_{4,5}, \text{with bullets} \mid G)$$

$$\beta_{NP}(4,5) \text{ for "the building"} = P(\text{the building} \mid NP_{4,5}, G)$$



Inside Probabilities: Base Step

$$\beta_j(p,q) = P(w_{pq}|N^j{}_{pq}, G)$$

Base case

$$\beta_j(k,k) = P(w_{kk}|N^j{}_{kk}, G)$$
$$= P(N^j \rightarrow w_k|G)$$

Base case for pre-terminals only

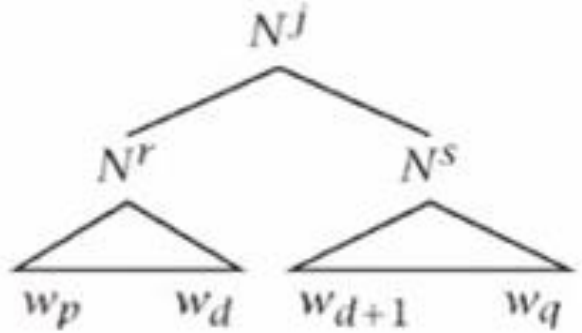
E.g., suppose $N^j = NN$ is being considered and $NN \rightarrow building$ is one of the rules with probability 0.5

$\beta_{NN}(5,5) = P(building|NN_{5,5}, G) = P(NN_{5,5} \rightarrow building|G)$

Inside Probabilities: Induction Step

Assuming Chomsky Normal Form, the first rule must be of the form $N^j \rightarrow N^r N^s$

$$\beta_j(p,q) = \sum_{r,s} \sum_{d=p}^{q-1} P(N^j \rightarrow N^r N^s) \beta_r(p,d) \beta_s(d+1,q)$$



- Consider different splits of the words - indicated by d
  E.g., *the huge building*
- Consider different non-terminals to be used in the rule:
  E.g., $NP \rightarrow DT\ NN$, $NP \rightarrow DT\ NNS$

Outside Probabilities

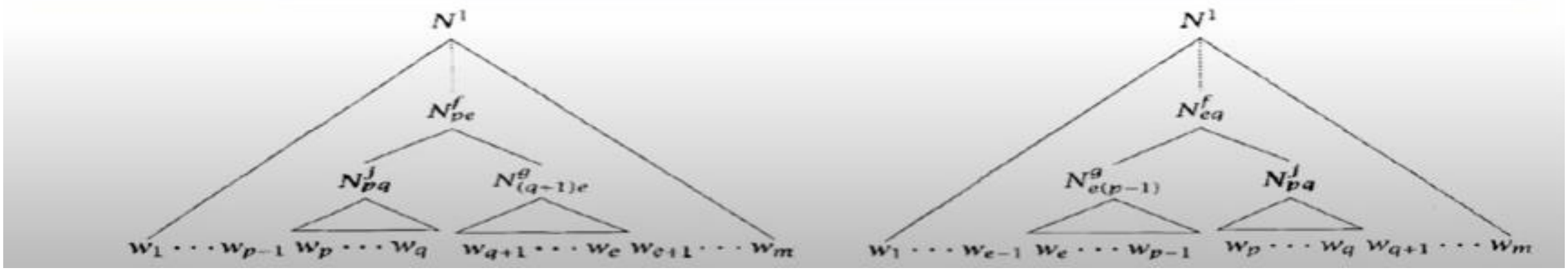
Compute top-down (after inside probabilities)

Base Case

$$\alpha_1(1,m) = 1$$

$$\alpha_j(1,m) = 0, j \neq 1$$

Induction

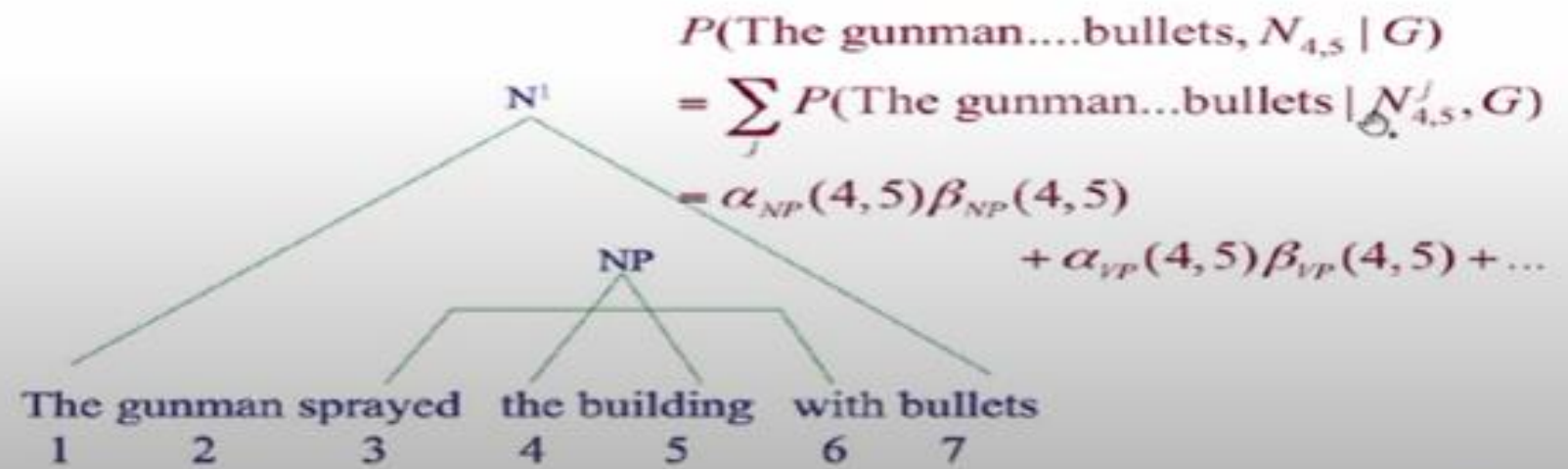


Product of inside-outside probabilities

$$\alpha_j(p,q)\beta_j(p,q) = P(w_{1(p-1)}, N^j_{pq}, w_{(q+1)m}|G)P(w_{pq}|N^j_{pq}, G) = P(w_{1m}, N^j_{pq}|G)$$

The probability of the sentence and that there is some consistent spanning from word p to q is given by:

$$P(w_{1m}, N_{pq}|G) = \sum \alpha_j(p,q)\beta_j(p,q) = P(N_1 \rightarrow w_{1m}, N_{pq} \rightarrow w_{pq}|G)$$

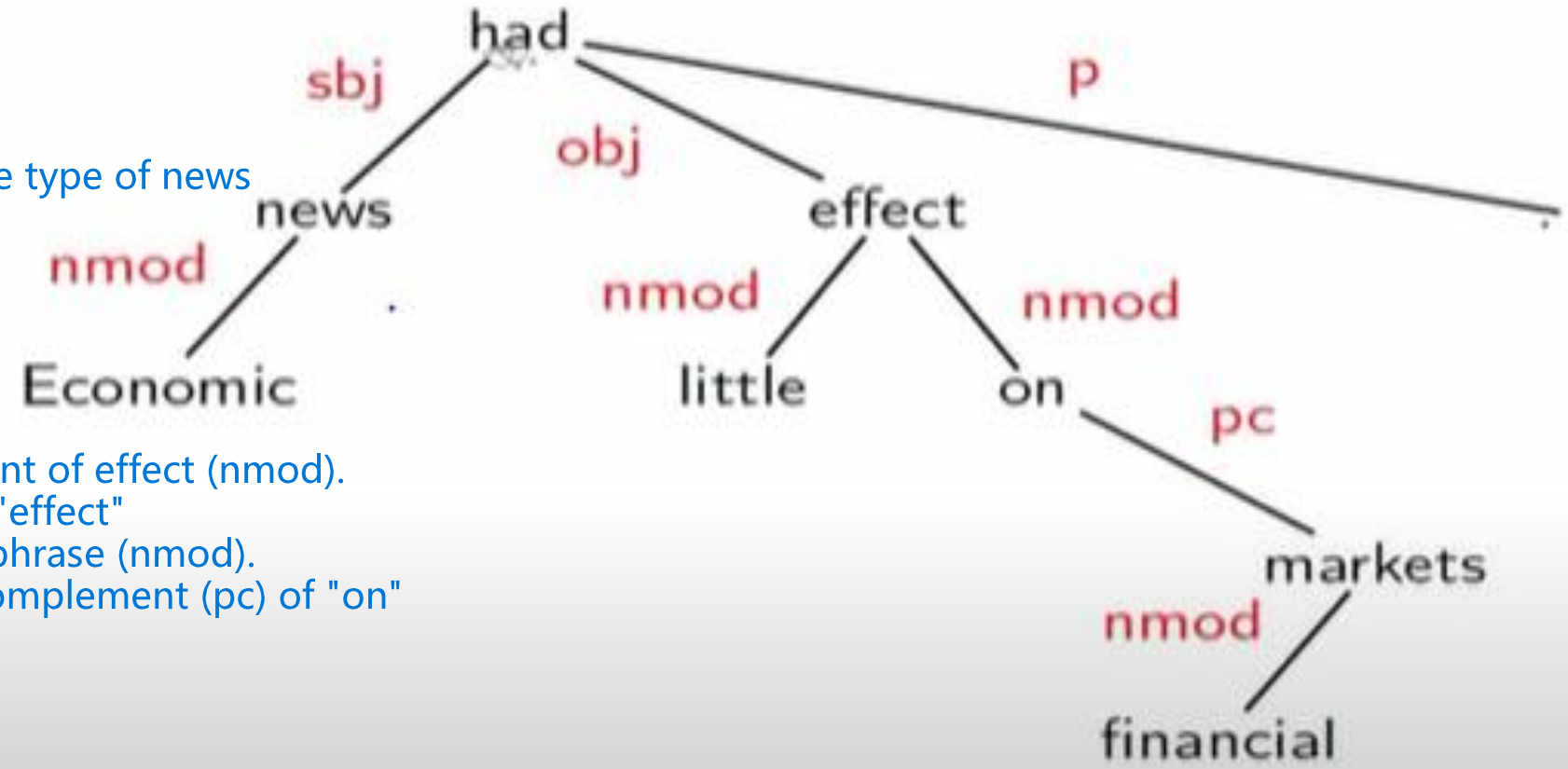


Dependency Structure Representation

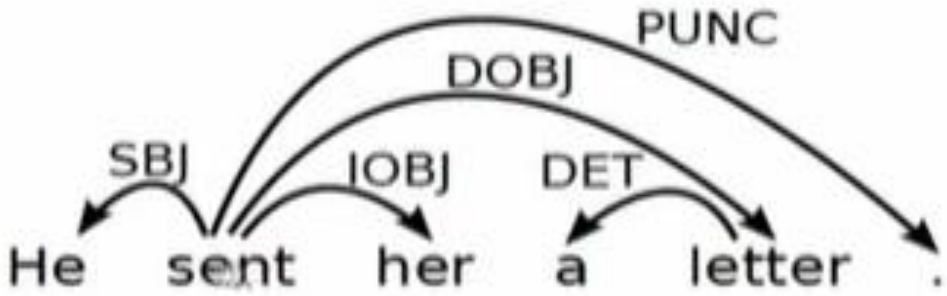
🧠 Core Structure
Root verb: "had" is the main verb of the sentence
— everything else connects to it.

🧩 Subject (sbj)
"news" is the subject of "had"
— it's what had something.
"Economic" modifies "news"
→ it's an adjective describing the type of news
(nmod = nominal modifier).

🎯 Object (obj)
"effect" is the object of "had"
— it's what was had.
"little" modifies "effect"
→ it tells us the degree or amount of effect (nmod).
"on" is a preposition modifying "effect"
→ it introduces a prepositional phrase (nmod).
"markets" is the prepositional complement (pc) of "on"
— it's the thing being affected.
"financial" modifies "markets"
→ it's an adjective (nmod).



Dependency Structure



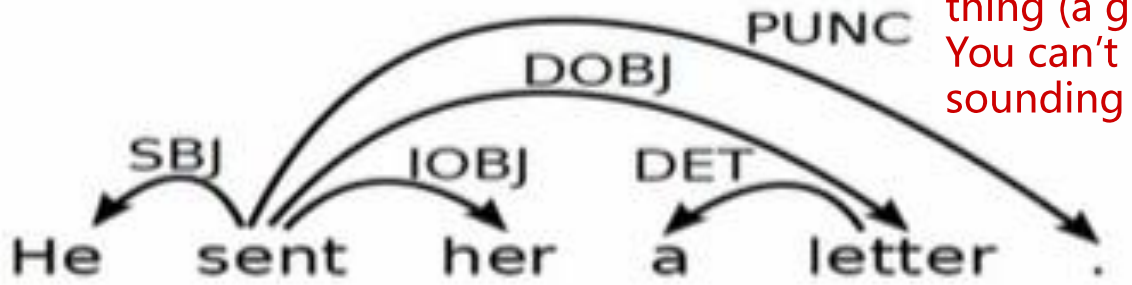
- Connects the words in a sentence by putting arrows between the words.
- Arrows show relations between the words and are typed by some grammatical relations.
- Arrows connect a head (governor, superior, regent) with a dependent (modifier, inferior, subordinate).
- Usually dependencies form a tree.

"House Detail Bonus Crew Dress Seat"

Criteria for Heads and Dependents

1. Example: "He [sent her a letter]."
The verb "sent" is the head of the verb phrase (VP).
You can replace the whole VP with "sent": "He sent."

2. Example: "She read [a book]."
"a book" (D) tells us what she read – it narrows down the meaning of "read".



4. Example: "She gave [him a gift]."
"gave" requires a recipient (him) and a thing (a gift).
You can't say "She gave." without sounding incomplete.

5. Example: "He likes [her]."
You say "her" (object form) not "she", because "likes" governs the case.

Criteria for a syntactic relation between a head H and a dependent D in a construction C

- *H* determines the syntactic category of *C*; *H* can replace *C*.
- *D* specifies *H*.
- *H* is obligatory; *D* may be optional.
- *H* selects *D* and determines whether *D* is obligatory.
- The form of *D* depends on *H* (agreement or government).
- The linear position of *D* is specified with reference to *H*.

3. Example: "He slept [peacefully]."
"slept" is required for the sentence to make sense.
"peacefully" (D) is optional — "He slept." is still valid.

6. Example: "She wrote [a poem]." "a poem" comes after the verb "wrote" — its position is determined by H.

Some Clear Cases

These are not headed by a single word that represents the whole phrase. The phrase's category isn't determined by any one word.

These do have a clear head that determines the category of the phrase.

Construction	Head	Dependent
Exocentric	Verb	Subject (sbj)
	Verb	Object (obj)
Endocentric	Verb	Adverbial (vmod)
	Noun	Attribute (nmod)

Dependency Graphs

- A dependency structure can be defined as a directed graph G, consisting of
  - a set V of nodes,
  - a set A of arcs (edges),
- Labeled graphs:
  - Nodes in V are labeled with word forms (and annotation).
  - Arcs in A are labeled with dependency types.
- Notational convention:
  - Arc (w_i, d, w_j) links head w_i to dependent w_j with label d
  - $w_i \xrightarrow{d} w_j \Leftrightarrow (w_i, d, w_j) \in A$
  - $i \rightarrow j \equiv (i, j) \in A$
  - $i \rightarrow^* j \equiv i = j \vee \exists k : i \rightarrow k, k \rightarrow^* j$

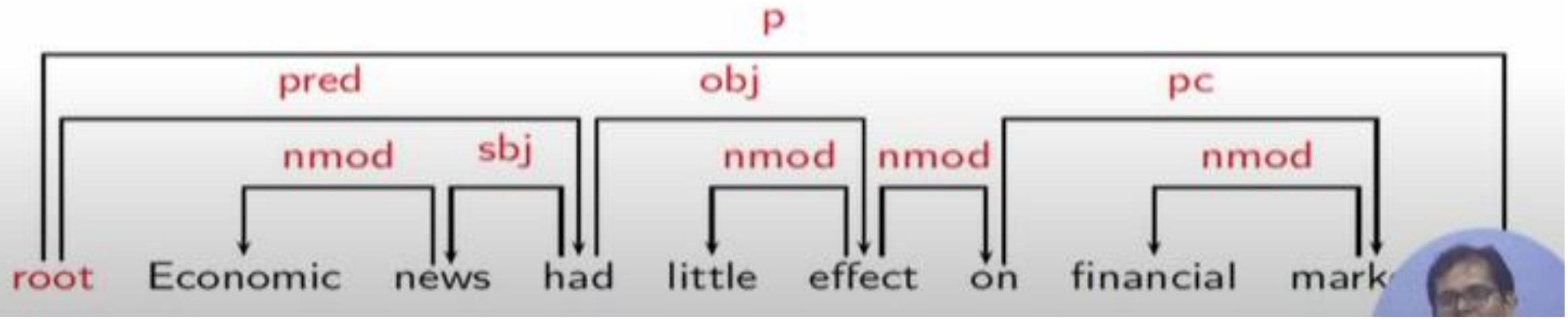
Formal conditions on Dependency Graphs

- G is connected:
  - For every node i there is a node j such that $i \rightarrow j$ or $j \rightarrow i$.
- G is acyclic:
  - if $i \rightarrow j$ then not $j \rightarrow^* i$.
- G obeys the single head constraint:
  - if $i \rightarrow j$ then not $k \rightarrow j$, for any $k \neq i$.
- G is projective:
  - if $i \rightarrow j$ then $j \rightarrow^* k$, for any k such that both j and k lie on the same side of i.

Formal Conditions: Basic Intuitions

Connectedness, Acyclicity and Single-Head

- **Connectedness:** Syntactic structure is complete.
- **Acyclicity:** Syntactic structure is hierarchical.
- **Single-Head:** Every word has at most one syntactic head.
- **Projectivity:** No crossing of dependencies.



Dependency Parsing

Deterministic → Rule-based

MST → Graph optimization

Constraint → Grammar-driven

Dependency Parsing

- **Input:** Sentence $x = w_1, \ldots, w_n$
- **Output:** Dependency graph G

Parsing Methods

- Deterministic Parsing
- Maximum Spanning Tree Based
- Constraint Propagation Based

1⃣ Deterministic Parsing
Uses a fixed set of rules or transitions.

Fast and efficient, but may struggle with ambiguity.

Example: Shift-reduce parsers.

2⃣ Maximum Spanning Tree (MST) Based
Treats parsing as a graph optimization problem.

Finds the best tree structure that connects all words with maximum score.

Often used in graph-based parsers.

3⃣ Constraint Propagation Based
Applies linguistic constraints (e.g., agreement, word order) to guide parsing.

Useful for enforcing grammatical rules and filtering invalid structures.